

```

// --- CORE SYSTEM FILES ---
// server.js
require('dotenv').config();
const express = require('express');
const session = require('express-session');
const cors = require('cors');
const path = require('path');
const fs = require('fs');
const multer = require('multer');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const { pool, initDatabase, tenantStore } = require('./db_postgres');
const bcrypt = require('bcryptjs');
const smsService = require('./sms_service');
const emailService = require('./email_service');
const ce = require('./crypto_envelope');
const lis = require('./lis');
const fe = require('./finance_engine');
const bbCompat = require('./bloodbank_compat');
const obEngine = require('./ob_engine');
const { insertSampleData, populateLabCatalog, populateRadiologyCatalog } =
require('./seed_data_pg');
const { populateMedicalServices, populateBaseDrugs } = require('./seed_services_pg');
const { addExtraLabTests, addExtraRadiology } = require('./seed_extra_catalog');
const { mountOrderRoutes } = require('./orders');
const { makeRequirePermission } = require('./rbac');
const { makeAuditMiddleware } = require('./audit_middleware');
const { validateBody } = require('./validation');
const RS = require('./route_schemas');
const { validatePasswordPolicy } = require('./password_policy');
const cds = require('./cds');
const { mountClinicalRoutes } = require('./clinical_cpoe');
const nursingScores = require('./nursing_scores');
const esiEngine = require('./esi_engine');
const icuScoring = require('./icu_scoring');
const specialtyScores = require('./specialty_scores');
const ewsEngine = require('./ews_engine');
const resultLoop = require('./result_loop');
const { resolveTenantContext } = require('./tenant_resolve');
const { makeIdempotencyGuard } = require('./idempotency');
const e11Engine = require('./e11_insurance_engine');
const pathologyEngine = require('./pathology_engine');
const e16 = require('./e16_inventory_engine');
const e18 = require('./e18_hr_engine');
const rareEngine = require('./rare_specialized_engine');
const imWave1Engine = require('./internal_medicine_wave1_engine');
const imWave1Engine2 = require('./internal_medicine_wave1_engine_batch2');
const imWave1Engine3 = require('./internal_medicine_wave1_engine_batch3');
const imWave1Engine4 = require('./internal_medicine_wave1_engine_batch4');
const surgWave2Engine = require('./surgical_wave2_engine');
const cardiologyEngine = require('./engines/cardiology_engine');
const icEngine = require('./engines/interventional_cardiology_engine');
const aiCardiology = require('./ai_cardiology_orchestrator');

```

```

const pulmonologyEngine = require('./engines/pulmonology_engine');
const gastroEngine = require('./engines/gastro_engine');
const surgeryEngine = require('./engines/surgery_engine');
const obgynEngine = require('./engines/obgyn_engine');
const obgynPedsWave3Engine = require('./obgyn_peds_wave3_engine');
const diagWave4Engine = require('./diagnostics_wave4_engine');
const critCareWave5Engine = require('./critical_care_wave5_engine');
const { mountOnboardingRoutes } = require('./onboarding');
const paymentAdapter = require('./payment_adapter');

// db_postgres.js
const { Pool } = require('pg');
const path = require('path');
const pool = new Pool({
  host: process.env.DB_HOST || 'localhost',
  port: parseInt(process.env.DB_PORT) || 5432,
  database: process.env.DB_NAME || 'nama_medical_web',
  user: process.env.DB_USER || 'postgres',
  password: process.env.DB_PASSWORD || 'postgres',
  max: parseInt(process.env.DB_MAX_CONNECTIONS) || 20,
  idleTimeoutMillis: process.env.NODE_ENV === 'production' ? 30000 : 1000
});

// tenant_resolve.js
function resolveTenantContext({ headerTenant, sessionUser, isProduction } = {}) {
  const sessionTenant = sessionUser ? positiveInt(sessionUser.tenantId) : null;
  if (sessionTenant !== null) {
    const facilityId = sessionUser && sessionUser.facilityId !== null ?
sessionUser.facilityId : null;
    return { tenantId: sessionTenant, facilityId, isProduction: !!isProduction,
source: 'session' };
  }
  if (sessionUser) {
    const headerT = positiveInt(headerTenant);
    if (headerT !== null) {
      return { tenantId: headerT, facilityId: null, isProduction: !!isProduction,
source: 'header-privileged' };
    }
    return { tenantId: null, facilityId: null, isProduction: !!isProduction, source:
'none' };
  }
  if (!isProduction) {
    return { tenantId: 1, facilityId: 1, isProduction: false, source: 'dev-fallback'
};
  }
  return { tenantId: null, facilityId: null, isProduction: true, source: 'none' };
}

// rbac.js
function makeRequirePermission(deps) {
  const { pool, getRequestTenantContext, roleFallback } = deps || {};
  return function requirePermission(key) {
    return async function (req, res, next) {
      try {

```

```

    if (!req.session || !req.session.user) {
      return res.status(401).json({ error: 'Unauthorized' });
    }
    const role = req.session.user.role;
    if (ADMIN_ROLES.has(role)) return next();
    const { tenantId } = getRequestTenantContext(req);
    const { rows } = await pool.query(
      'SELECT permission_key FROM role_permissions WHERE role = $1 AND ($2::int IS NULL OR tenant_id = $2)',
      [role, tenantId || null]
    );
    if (rows.length === 0) {
      if (typeof roleFallback === 'function') {
        return roleFallback(req) ? next() : res.status(403).json({
          error: 'Access denied' });
      }
      return res.status(403).json({ error: 'Access denied' });
    }
    const granted = rows.some(r => r.permission_key === key);
    if (granted) return next();
    return res.status(403).json({ error: 'Access denied' });
  } catch (e) {
    return res.status(403).json({ error: 'Access denied' });
  }
};
};
}

```